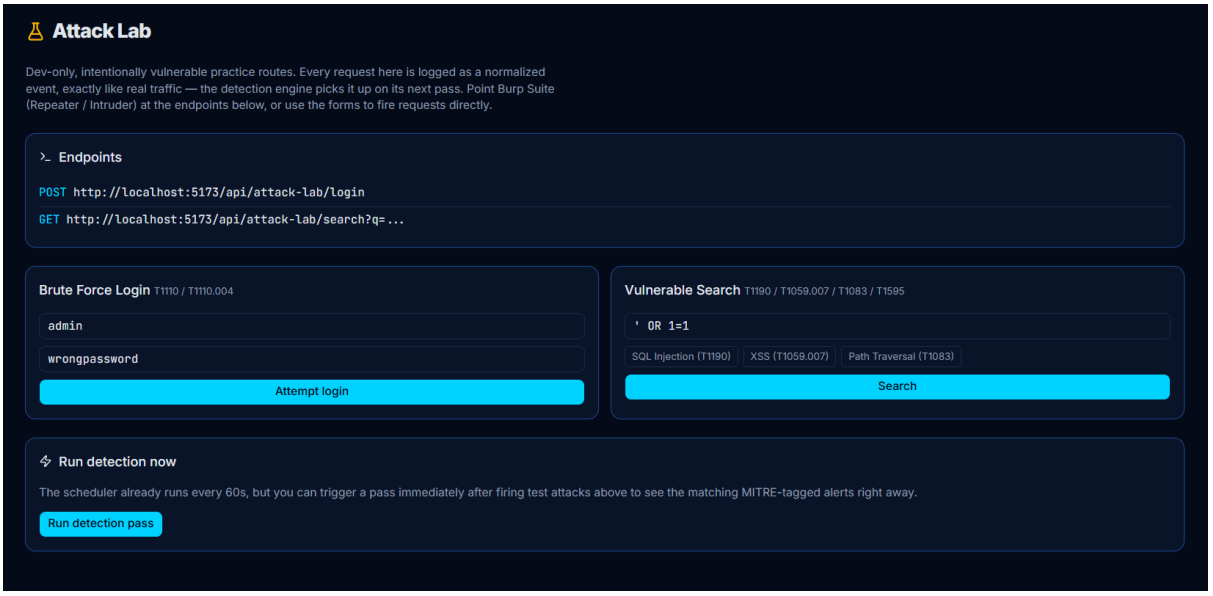
Three simulated attacks fired at my own Mini SIEM, then investigated from the alerts and evidence the system recorded.

Environment

- **Tool:** Mini SIEM, local development build, with the dev-only Attack Lab enabled
- **Date:** 17 October 2026
- **Source IP in all alerts:** `::1` (my own machine, localhost). These are simulated attacks from the same computer, not outside traffic.
- **Detection:** the engine runs a pass every 60 seconds, and the Attack Lab also has a manual "Run detection pass" button
- **Correlation:** alerts from the same source within 60 minutes are grouped into one incident ("campaign")



Summary

#	Attack	Time	Alert	MITRE	Score / Severity	Verdict
1	Brute force login	6:44 PM	Brute force login attempts from ::1	T1110	30 / HIGH	True positive (simulated)
2	SQL injection	6:50 PM	SQL Injection Attempt in HTTP Request	T1190	25 / HIGH	True positive (simulated)
3	Path traversal	6:56 PM	Path Traversal Attempt in HTTP Request	T1083	20 / HIGH	True positive (simulated)

All three alerts were correlated into one incident, "Attack campaign from ::1".

Incident 1: Brute-force login attempts

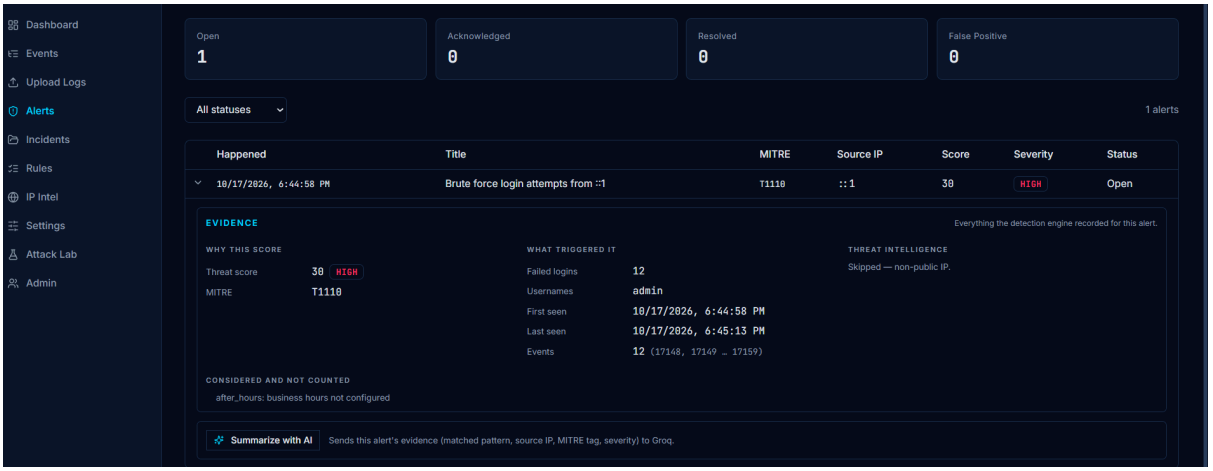
Test: 12 login attempts for the user `admin` with a wrong password, started 17 Oct 2026, 6:44 PM.

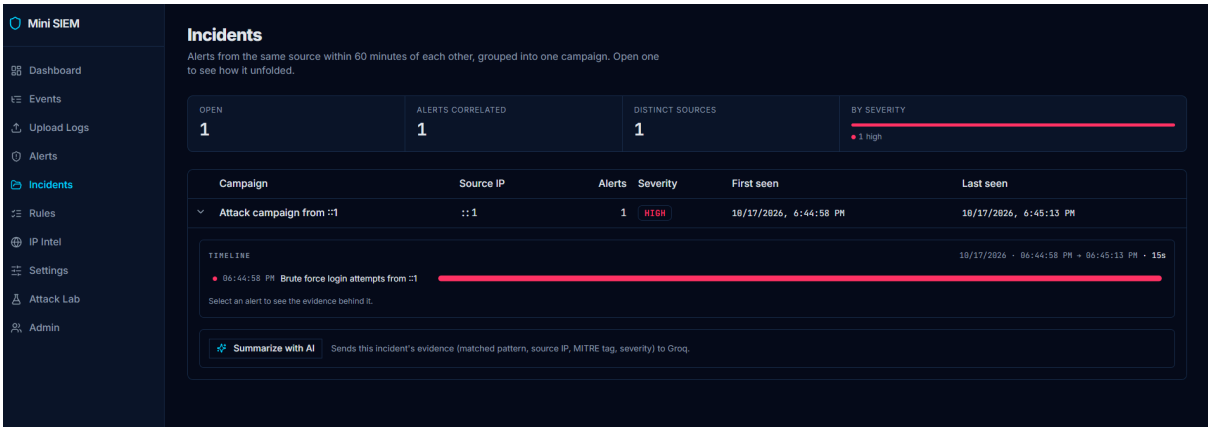
What the system recorded

- Alert: *Brute force login attempts from ::1*, MITRE T1110, threat score 30, HIGH, status Open
- 12 failed logins for the username `admin`
- First seen 6:44:58 PM, last seen 6:45:13 PM (15 seconds)
- 12 events (IDs 17148 to 17159)

Timeline

Time	Event
6:44:58 PM	First failed login for `admin` from `::1`
6:45:13 PM	Twelfth failed login, last event in the burst
After 6:45:13 PM	Alert raised and an incident created with 1 alert





Verdict: True positive. Twelve failures against one username in 15 seconds is far faster than a person mistyping a password, so it fits password guessing. It was a deliberate test.

Response I would take on a real alert: check whether any login for `admin` succeeded afterwards, block or rate-limit the source IP, review other accounts hit from the same source, and consider forcing a password reset if there was a success.

Incident 2: SQL injection

Test: the search form was sent the payload `' OR 1=1`, at 17 Oct 2026, 6:50 PM.

What the system recorded

- Alert: *SQL Injection Attempt in HTTP Request*, MITRE T1190, threat score 25, HIGH, status Open
- Matched field: `url`, matched value `/search?q=' OR 1=1`
- Matched patterns: `' OR 1=1` and `OR 1=1`
- 1 matching request (event ID 17160), first and last seen 6:50:55 PM

Timeline

Time	Event
6:50:55 PM	Request to `/search` containing `' OR 1=1` from `::1`
After 6:50:55 PM	Alert raised and added to the existing campaign (2 alerts, 6 minutes after the first)







Verdict: True positive. `' OR 1=1` is a textbook injection string meant to make a database condition always true.

Response I would take on a real alert: check the web and database logs for what the request returned, confirm whether the search input is parameterised (so the payload can't change the query), block the source IP, and look for similar requests from other sources.

Incident 3: Path traversal

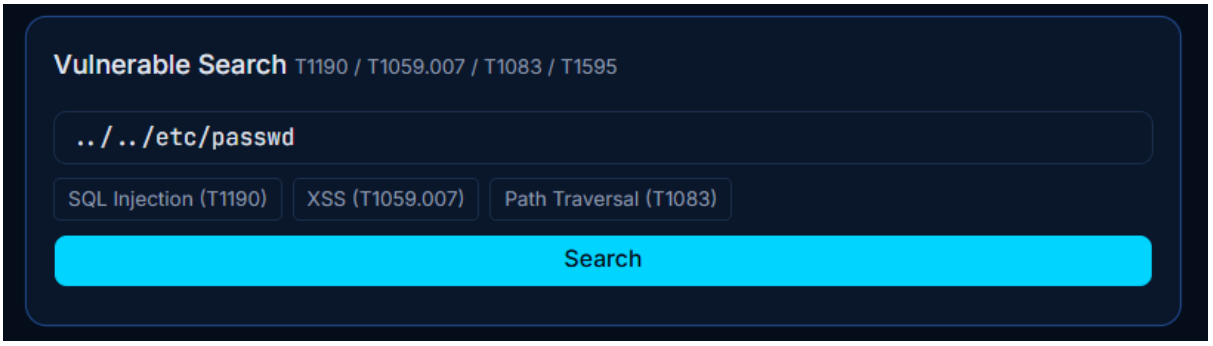
Test: the search form was sent the payload `../../etc/passwd`, at 17 Oct 2026, 6:56 PM.

What the system recorded

- Alert: *Path Traversal Attempt in HTTP Request*, MITRE T1083, threat score 20, HIGH, status Open
- Matched field: `url`, matched value `/search?q=../../etc/passwd`
- Matched patterns: `../` and `/etc/passwd`
- 1 matching request (event ID 17161), first and last seen 6:56:59 PM

Timeline

Time	Event
6:56:59 PM	Request to `/search` containing `../../etc/passwd` from `::1`
After 6:56:59 PM	Alert raised and shown in the same campaign timeline





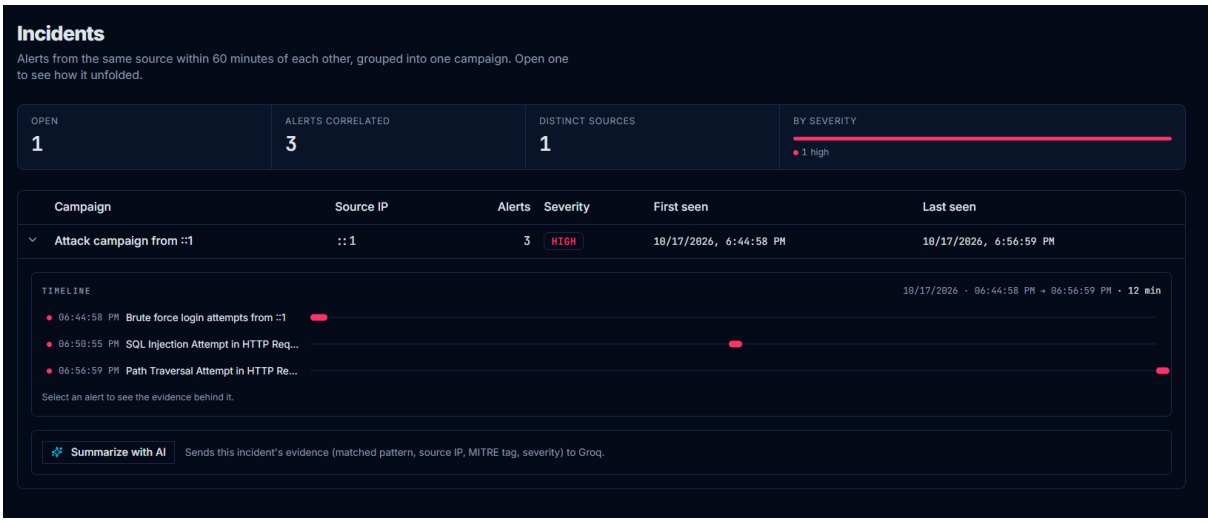


Verdict: True positive. The request tries to climb out of the web folder and read the Linux password file.

Response I would take on a real alert: check whether the server returned file contents, make sure file paths from user input are validated or blocked, block the source IP, and review other requests from the same source.

Correlation result

All three attacks came from the same source within 60 minutes, so the system merged them into one HIGH incident, "Attack campaign from ::1". Instead of three unrelated alerts, an analyst sees one story: password guessing first, then injection, then file access attempts.



Detection gaps and limitations noticed

- **Localhost source:** every alert came from `::1`, so geolocation and reputation features were not exercised.
- **Threat intelligence skipped:** the alert evidence shows threat intelligence was skipped because the IP was non-public, so the scores come from rule matches only.
- **Business-hours scoring not configured:** the evidence shows "after_hours: business hours not configured", so time-of-day context added nothing to the scores.
- **Single-request detections:** the SQL injection and path traversal alerts were each triggered by one request, so one probe is enough to raise a HIGH alert. In a real environment that could create noise if scanners hit the site often.
- **Simulated data:** these results show that the detection rules work against test attacks I generated, not against real-world traffic.

What I learned

- How alerts, evidence and correlation fit together, from a raw log event to an incident
- How to read a detection alert: the matched field, matched patterns, score and MITRE technique
- Why context (source IP, threat intelligence, business hours) matters when judging an alert